

Procedures

Chapter-5

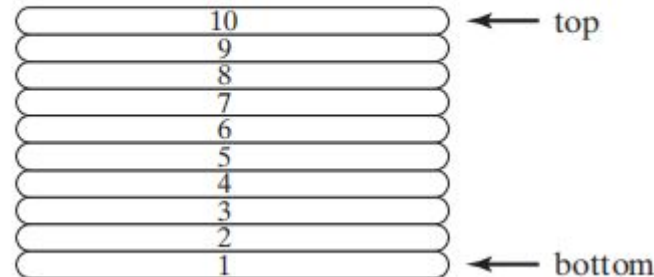
Stack Operation

If we place 10 plates on each other as in the following diagram, the result can be called a stack.

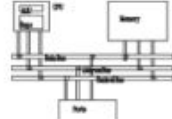
A *stack data structure* follows the same principle as a stack of plates: New values are added to the top of the stack, and existing values are removed from the top.

A stack is also called a **LIFO** structure (*Last-In, First-Out*) because the last value put into the stack is always the first value taken out.

Stack of Plates.



Stacks



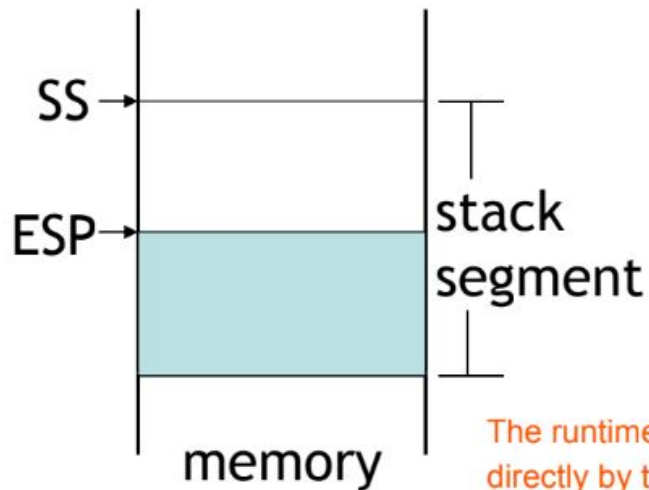
- LIFO (Last-In, First-Out) data structure.
- push/pop operations
- You probably have had experiences on implementing it in high-level languages.
- Here, we concentrate on ***runtime stack***, directly supported by hardware in the CPU. It is essential for calling and returning from procedures.

The runtime stack stores information about the active subroutines of a computer program.

Runtime stack



- Managed by the CPU, using two registers
 - SS (stack segment)
 - ESP (stack pointer) * : point to the top of the stack usually modified by **CALL**, **RET**, **PUSH** and **POP**



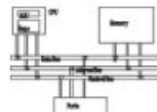
The runtime stack is a memory array managed directly by the CPU, using the ESP register, known as the stack pointer register.

* SP in Real-address mode

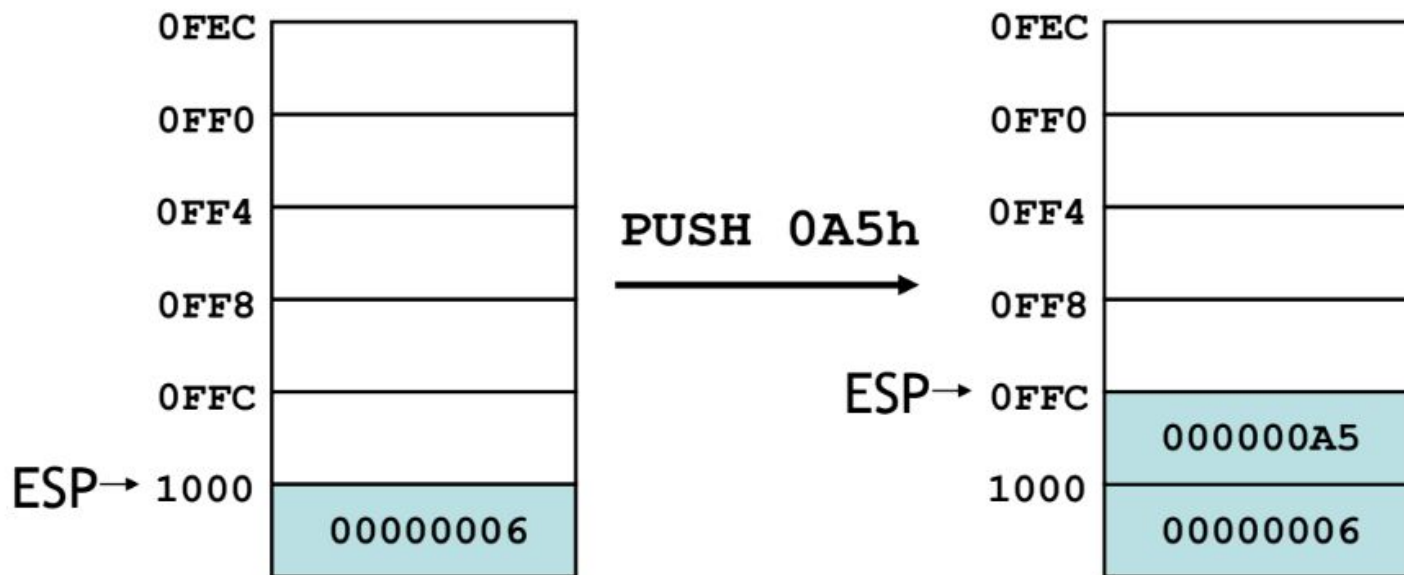
PUSH and POP instructions

- **PUSH** syntax:
 - `PUSH r/m16`
 - `PUSH r/m32`
 - `PUSH imm32`
- **POP** syntax:
 - `POP r/m16`
 - `POP r/m32`

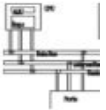
PUSH operation (1 of 2)



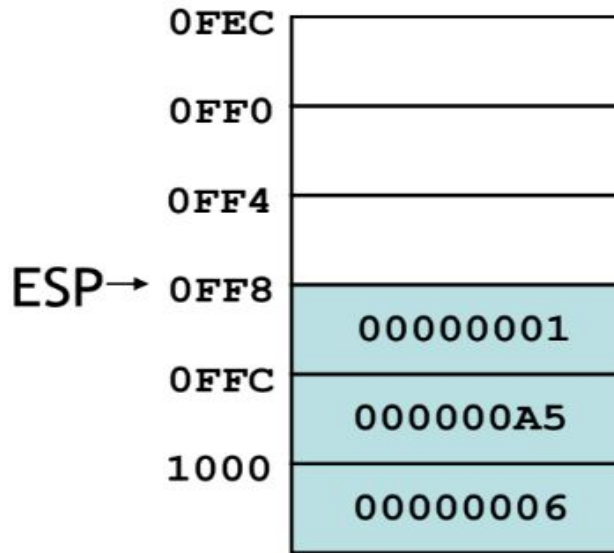
- A **push** operation decrements the stack pointer by 2 or 4 (depending on operands) and copies a value into the location pointed to by the stack pointer.



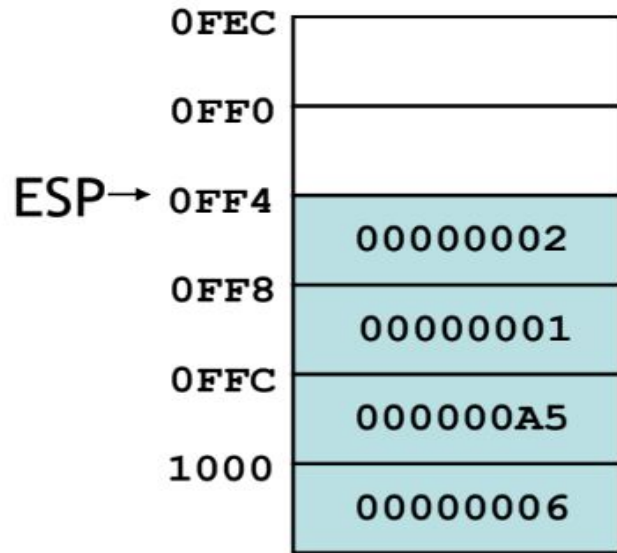
PUSH operation (2 of 2)



- The same stack after pushing two more integers:

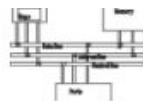


PUSH 01h

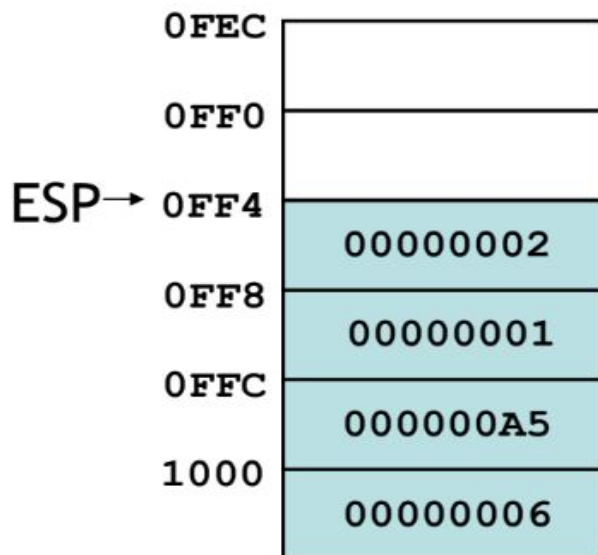


PUSH 02h

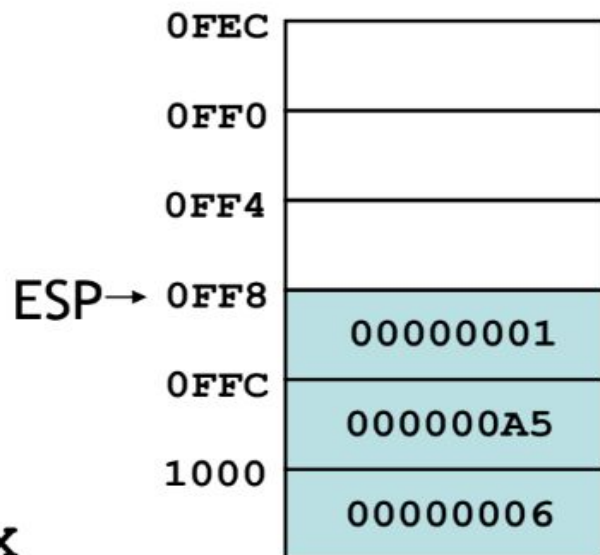
POP operation



- Copies value at stack[ESP] into a register or variable.
- Adds n to ESP, where n is either 2 or 4, depending on the attribute of the operand receiving the data



POP EAX



EAX=00000002

Stack Applications

- When calling a subroutine, you pass input values called *arguments* by pushing them on the stack.
- The stack provides temporary storage for local variables inside subroutines.
- A stack makes a convenient than one purpose. After they are modified, they can be restored to their original values.(for registers etc.)
- When the CALL instruction executes, the CPU saves the current subroutine's return address on the stack.
- Application which is LIFO in nature such as Reversing a String

Stack Applications

Use Case	What the Stack Does	Key Instructions
Passing Parameters	Stores function arguments	PUSH , CALL , RET
Local Variables	Holds temporary variables per call	SUB ESP, n
Saving Registers	Saves original values before changes	PUSH , POP
Recursive Calls	Maintains separate stack frames	CALL , RET
Temporary Storage	Holds data when registers are full	PUSH , POP

Passing Parameters to Functions

When calling a procedure, parameters are often **pushed** onto the stack.

```
.data
result DWORD ?

.code
main PROC
    PUSH 6          ; Parameter 2
    PUSH 4          ; Parameter 1
    CALL AddNums    ; Call procedure
    MOV result, EAX
    exit
main ENDP
```

```
AddNums PROC
    PUSH EBP
    MOV EBP, ESP
    MOV EAX, [EBP+8] ; Get first parameter (4)
    ADD EAX, [EBP+12] ; Add second parameter (6)
    POP EBP
    RET 8            ; Clean 2 parameters (8 bytes)
AddNums ENDP

END main
```

Stack used to **pass parameters** and **store return address**.

Storing Local Variables

Local variables are kept **on the stack** so that each procedure call has its own copy.

```
LocalVars PROC
    PUSH EBP
    MOV EBP, ESP
    SUB ESP, 8           ; Reserve 8 bytes for 2 local variables

    MOV DWORD PTR [EBP-4], 5    ; local1 = 5
    MOV DWORD PTR [EBP-8], 10   ; local2 = 10
    ADD EAX, [EBP-4]
    ADD EAX, [EBP-8]

    MOV ESP, EBP          ; Deallocate locals
    POP EBP
    RET

LocalVars ENDP
```

Stack provides **temporary storage space** for local variables.

Saving Registers Before Modification

Before using registers that hold important data, we can save them on the stack.

```
push esi                ; push registers
push ecx
push ebx

mov esi,OFFSET dwordVal ; starting OFFSET
mov ecx,LENGTHOF dwordVal; number of units
mov ebx,TYPE dwordVal ;size of a doubleword
call DumpMem            ; display memory

pop ebx                 ; opposite order
pop ecx
pop esi
```

Stack protects original register values — useful in subroutines or interrupt routines.

Handling Nested or Recursive Function Calls

Each recursive call creates a **new stack frame**, allowing unique local variables and parameters.

```
Factorial PROC
```

```
    PUSH EBP
```

```
    MOV EBP, ESP
```

```
    MOV EAX, [EBP+8]    ; n
```

```
    CMP EAX, 1
```

```
    JBE BaseCase
```

```
    DEC EAX
```

```
    PUSH EAX            ; Push (n-1)
```

```
    CALL Factorial      ; Recursive call
```

```
    MOV EBX, [EBP+8]
```

```
    IMUL EAX, EBX
```

```
    JMP Done
```

```
BaseCase:
```

```
    MOV EAX, 1
```

```
Done:
```

```
    POP EBP
```

```
    RET 4
```

```
Factorial ENDP
```

Each recursive call **pushes parameters and return address**, maintaining separate stack frames.

Handling Nested or Recursive Function Calls

When creating a nested loop, push the outer loop counter before entering the inner loop:

```
    mov ecx,100      ; set outer loop count
L1:      ; begin the outer loop
    push ecx        ; save outer loop count

    mov ecx,20       ; set inner loop count
L2:      ; begin the inner loop
    ;
    ;
    loop L2          ; repeat the inner loop

    pop ecx          ; restore outer loop count
    loop L1          ; repeat the outer loop
```

Temporary Data Storage During Calculations

You can use the stack as a **scratchpad** for temporary values.

```
MOV AX, 5
MOV BX, 10
PUSH AX      ; Temporarily save AX
MOV AX, BX   ; AX = 10
POP BX       ; BX = 5 (restored from stack)
```

Stack acts as a **temporary holding area** when registers are limited.

Reversing a String

```
.data
str1 BYTE "FAST University", 0           ; Original string
len  DWORD ($ - str1 - 1)                ; String length (exclude null terminator)
revStr BYTE 50 DUP(?)                    ; Space for reversed string

.code
main PROC
    mov ecx, len                          ; Counter = string length
    mov esi, OFFSET str1                  ; Source pointer (original string)

push_loop:
    mov al, [esi]                          ; Load character
    push eax                              ; Push onto stack
    inc esi
    loop push_loop
```

Reversing a String

Pop each character to new string (reversed)

```
mov ecx, len
mov edi, OFFSET revStr ; Destination pointer (reversed string)

pop_loop:
    pop eax
    mov [edi], al        ; Store character in reversed string
    inc edi
    loop pop_loop

    mov BYTE PTR [edi], 0 ; Null-terminate the reversed string

    exit
main ENDP

END main
```

PUSHFD and POPFD Instructions

The PUSHFD instruction pushes the 32-bit EFLAGS register on the stack, and POPFD pops the stack into EFLAGS:

```
pushfd  
popfd
```

16-bit programs use the PUSHF instruction to push the 16-bit FLAGS register on the stack and POPF to pop the stack into FLAGS.

```
pushfd                ; save the flags  
;  
; any sequence of statements here...  
;  
popfd                ; restore the flags
```

PUSHAD, PUSHA, POPAD, AND

- **PUSHAD** pushes the 32-bit general-purpose registers on the stack in the following order
 - **EAX, ECX, EDX, EBX, ESP, EBP, ESI, EDI**
- **POPAD** pops the same registers off the stack in reverse order
 - **PUSHA** and **POPA** do the same for 16-bit registers

```
MySub PROC
    pushad                ; save general-purpose registers
    .
    .
    mov eax, ...
    mov edx, ...
    mov ecx, ...
    .
    .
    popad                 ; restore general-purpose registers
    ret
MySub ENDP
```

Example

```
MySub PROC  
    pushad  
    ...  
    ; modify some register  
    ...  
    popad  
    ret  
MySub ENDP
```

Do not use this if your procedure uses registers for return values

Your Turn . . .

- Write a program that does the following:
 - Assigns integer values to EAX, EBX, ECX, EDX, ESI, and EDI
 - Uses PUSHAD to push the general-purpose registers on the stack
 - Using a loop, your program should pop each integer from the stack and display it on the screen

```
INCLUDE Irvine32.inc
```

```
.data
```

```
msg BYTE "Register value: ", 0
```

```
.code
```

```
main PROC
```

```
    ; -----  
    ; Assign integer values to registers  
    ; -----
```

```
    mov eax, 10
```

```
    mov ebx, 20
```

```
    mov ecx, 30
```

```
    mov edx, 40
```

```
    mov esi, 50
```

```
    mov edi, 60
```

```
    ; -----
```

```

; -----
; Push all general-purpose registers
; -----
pushad          ; pushes: EAX, ECX, EDX, EBX, ESP, EBP, ESI, EDI

; -----
; Pop and display 8 values from stack
; -----
mov ecx, 8      ; total 8 registers pushed
display_loop:
    pop eax      ; get top value from stack
    mov edx, OFFSET msg
    call WriteString
    call WriteDec ; display value in EAX
    call Crlf
    loop display_loop

    exit
main ENDP
END main

```

 Microsoft Visual Studio Debug Console

Register value: 60

Register value: 50

Register value: 17823684

Register value: 17823672

Register value: 20

Register value: 40

Register value: 30

Register value: 10

Creating Procedures



- Large problems can be divided into smaller tasks to make them more manageable
- A procedure is the ASM equivalent of a Java or C++ function
- Following is an assembly language procedure named sample:

```
sample PROC  
.  
.  
ret  
sample ENDP
```

A named block of statements that ends with a return.

Documenting procedures



Suggested documentation for each procedure:

- A description of all tasks accomplished by the procedure.
- Receives: A list of input parameters; state their usage and requirements.
- Returns: A description of values returned by the procedure.
- Requires: Optional list of requirements called preconditions that must be satisfied before the procedure is called.

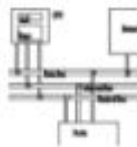
For example, a procedure of drawing lines could assume that display adapter is already in graphics mode.

Example: SumOf procedure



```
;-----  
SumOf PROC  
;  
; Calculates and returns the sum of three 32-bit  
; integers.  
; Receives: EAX, EBX, ECX, the three integers.  
;           May be signed or unsigned.  
; Returns: EAX = sum, and the status flags  
;           (Carry, Overflow, etc.) are changed.  
; Requires: nothing  
;-----  
    add eax,ebx  
    add eax,ecx  
    ret  
SumOf ENDP
```

CALL and RET instructions



- The **CALL** instruction calls a procedure
 - pushes offset of next instruction on the stack
 - copies the address of the called procedure into **EIP**
- The **RET** instruction returns from a procedure
 - pops top of stack into **EIP**

CALL-RET example (1 of 2)



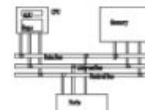
0000025 is the offset
of the instruction
immediately following
the CALL instruction

```
main PROC
    00000020 call MySub
    00000025 mov  eax,ebx
    .
    .
main ENDP
```

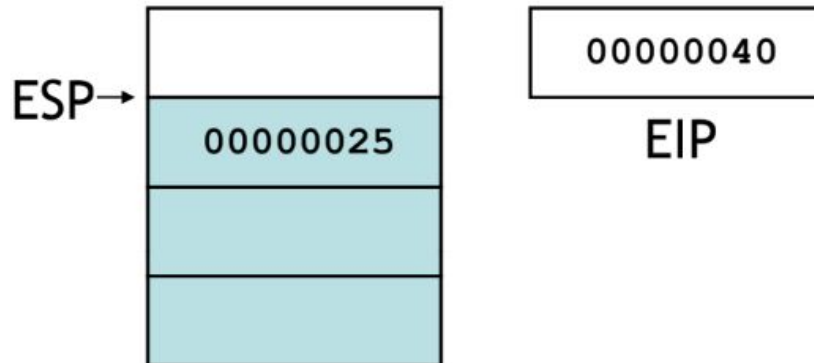
00000040 is the offset
of the first instruction
inside MySub

```
MySub PROC
    00000040 mov  eax,edx
    .
    .
    ret
MySub ENDP
```

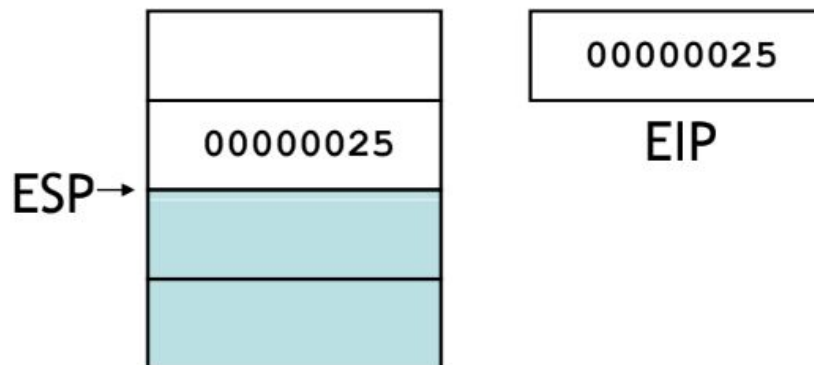
CALL-RET example (2 of 2)



The CALL instruction pushes 00000025 onto the stack, and loads 00000040 into EIP



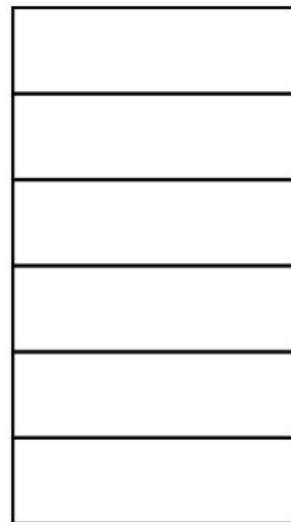
The RET instruction pops 00000025 from the stack into EIP



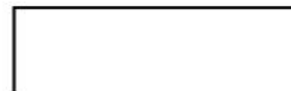
Nested procedure calls



```
0050  main PROC  
      .  
      .  
      call Sub1  
      exit  
      main ENDP  
0100  Sub1 PROC  
      .  
      .  
      call Sub2  
      ret  
      Sub1 ENDP  
0200  Sub2 PROC  
      .  
      .  
      call Sub3  
      ret  
      Sub2 ENDP  
0300  Sub3 PROC  
      .  
      .  
      ret  
      Sub3 ENDP
```



Stack



EIP

In assembly language, it is common to **pass arguments inside general-purpose** registers.

```
.data
theSum  DWORD  ?
.code
main PROC
    mov     eax,10000h           ; argument
    mov     ebx,20000h           ; argument
    mov     ecx,30000h           ; argument
    call    Sumof                ; EAX = (EAX + EBX + ECX)
    mov     theSum,eax           ; save the sum

SumOf PROC
    add     eax,ebx
    add     eax,ecx
    ret
SumOf ENDP
```


Local and Global Labels

A local label is visible only to statements inside the same procedure. A global label is visible everywhere.

```
main PROC
    jmp L2                                ; error
L1::                                     ; global label
    exit
main ENDP

sub2 PROC
L2:                                     ; local label
    jmp L1                               ; ok
    ret
sub2 ENDP
```

Procedure Parameters (1 of 3)

- A good procedure might be usable in many different programs
 - but not if it refers to specific variable names
- Parameters help to make procedures flexible because parameter values can change at runtime

Procedure Parameters (2 of 3)

The ArraySum procedure calculates the sum of an array. It makes two references to specific variable names:

```
ArraySum PROC
    mov esi,0                ; array index
    mov eax,0                ; set the sum to zero
    mov ecx,LENGTHOF myarray ; set number of elements

L1: add eax,myArray[esi]     ; add each integer to sum
    add esi,4                ; point to next integer
    loop L1                  ; repeat for array size

    mov theSum,eax           ; store the sum
    ret
ArraySum ENDP
```

What if you wanted to calculate the sum of two or three arrays within the same program?

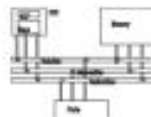
Procedure parameters (3 of 3)



This version returns the sum of any doubleword array whose address is in ESI. The sum is returned in EAX:

```
ArraySum PROC
; Recevies: ESI points to an array of doublewords,
;           ECX = number of array elements.
; Returns:  EAX = sum
;-----
    push esi
    push ecx
    mov eax,0                ; set the sum to zero
L1: add eax,[esi]             ; add each integer to sum
    add esi,4                ; point to next integer
    loop L1                  ; repeat for array size
    pop ecx
    pop esi
    ret
ArraySum ENDP
```

Calling ArraySum



```
.data
```

```
array DWORD 10000h, 20000h, 30000h, 40000h
```

```
theSum DWORD ?
```

```
.code
```

```
main PROC
```

```
    mov     esi, OFFSET array
```

```
    mov     ecx, LENGTHOF array
```

```
    call    ArraySum
```

```
    mov     theSum, eax
```

USES operator



- Lists the registers that will be saved (to avoid side effects) (return register shouldn't be saved)

```
ArraySum PROC USES esi ecx  
    mov eax,0    ; set the sum to zero  
    ...
```

MASM generates the following code:

```
ArraySum PROC  
    push esi  
    push ecx  
    .  
    .  
    pop ecx  
    pop esi  
    ret  
ArraySum ENDP
```